



Tronity

Bindet Fahrzeugdaten über den Tronity-Datenaggregator in Home Assistant ein. Tronity ist ein Drittanbieter-Dienst der sich selbst mit dem Fahrzeughersteller verbindet und die Daten über eine eigene API bereitstellt.

ev

vehicle

cloud

aggregator

Sicherheits-Übersicht

KANN ICH DAS INSTALLIEREN?

● Mit Einschränkungen verwendbar

Die Fahrzeugdaten laufen über einen zusätzlichen Drittanbieter: Tronity. Statt direkt mit dem Hersteller zu kommunizieren, gibt die Integration Client-ID und Client-Secret an Tronity weiter, das seinerseits auf die Fahrzeugdaten zugreift. Wer Tronity vertraut, kann die Integration bedenkenlos nutzen.

- Tronity ist ein Datenaggregator — Fahrzeugdaten laufen über deren Server in der Mitte
- Client-ID und Client-Secret werden HA-verschlüsselt gespeichert
- Keine offiziellen Releases auf GitHub — v0.2.0, potenziell wenig gewartet
- Manifest deklariert fälschlicherweise local_polling — es ist Cloud-Polling

TECHNISCHE INTEGRITÄT

● Technische Mängel vorhanden

Der Code ist funktional aber enthält handwerkliche Schwächen: das Auth-Token wird bei jedem einzelnen Poll neu abgerufen statt gecacht, und bei einem fehlgeschlagenen Token-Abruf läuft der Code stillschweigend mit einem ungültigen Token weiter. Das `iot_class`-Flag ist falsch gesetzt.

- Token-Abruf bei jedem Poll (60s) — kein Caching, unnötige Last auf Auth-Server
- Kein Fehler-Handling wenn Token-Abruf fehlschlägt — stille 401-Fehler möglich
- `iot_class: local_polling` im Manifest ist falsch — es ist Cloud-Polling
- Keine Versions-Releases — Wartungsstatus unklar

Security Findings

SCHWERE	KATEGORIE	FINDING	BETRIFFT
MEDIUM	Datenschutz	Fahrzeugdaten über Tronity als Zwischenstation	const.py — CONF_VEHICLES_URL: api.tronity.tech

Die Integration kommuniziert nicht direkt mit dem Fahrzeughersteller, sondern mit `api.tronity.tech`. Tronity ist ein Drittanbieter der sich mit verschiedenen Fahrzeugherstellern verbindet und die Daten aggregiert. Damit sehen zwei Parteien die Fahrzeugdaten: der Hersteller und Tronity. Tronity hat Einblick in Kilometerstand, Akkustand, Ladestand, Reichweite und GPS-Position.

Empfehlung: Datenschutzrichtlinie von Tronity prüfen. Nur verwenden wenn Tronity als Datenverarbeiter akzeptiert wird.

SCHWERE	KATEGORIE	FINDING	BETRIFFT
MEDIUM	Authentifizierung	Neues Bearer-Token bei jedem Poll-Durchlauf	__init__.py — async_update_data(), get_bearer_token()
Die Funktion <code>async_update_data()</code> ruft bei jedem Poll-Intervall (60 Sekunden) <code>get_bearer_token()</code> auf — ein vollständiger POST-Request an den Auth-Server. Das Token wird nie gecacht oder wiederverwendet. Das bedeutet: pro Stunde 60 Auth-Requests statt einem. Bei vielen Nutzern eine unnötige Last; bei Tronity-seitigem Rate-Limiting würden Datenupdates still ausfallen. Empfehlung: Token cachen und nur bei Ablauf erneuern. Tronity-Token-Ablaufzeit aus der API-Antwort (<code>expires_in</code>) auslesen.			
MEDIUM	Stabilität	Kein Fehler-Handling bei fehlgeschlagenem Token-Abruf	__init__.py — get_bearer_token() Zeile ~50
Wenn <code>get_bearer_token()</code> fehlschlägt (Netzwerkfehler, HTTP-Fehler, ungültige Credentials), gibt die Funktion <code>None</code> zurück. Die aufrufende Funktion setzt dann <code>headers = {'Authorization': 'Bearer None'}</code> und sendet den API-Request trotzdem ab — was zu einem stillen 401-Fehler führt, der als <code>UpdateFailed</code> protokolliert wird ohne klaren Hinweis auf die Ursache. Empfehlung: Rückgabewert von <code>get_bearer_token()</code> auf <code>None</code> prüfen und explizit <code>ConfigEntryAuthFailed</code> oder <code>UpdateFailed</code> werfen.			
LOW	Stabilität	iot_class im Manifest falsch deklariert	manifest.json — iot_class
<code>manifest.json</code> enthält <code>iot_class: 'local_polling'</code>. Die Integration kommuniziert ausschliesslich mit <code>api.tronity.tech</code> — es gibt keinen lokalen Gerätezugriff. Der korrekte Wert wäre <code>cloud_polling</code>. Das falsche Flag kann HA-Diagnosetools und Nutzer irreführen. Empfehlung: <code>iot_class</code> auf <code>'cloud_polling'</code> ändern.			
LOW	Stabilität	Keine offiziellen Releases — Wartungsstatus unklar	github.com/tronity/homeassistant — releases
Das GitHub-Repository hat keine veröffentlichten Releases. Version 0.2.0 steht nur im <code>manifest.json</code>. Es gibt keinen Changelog, kein Release-Datum und kein klares Signal ob die Integration aktiv gewartet wird. Bei Breaking Changes in der Tronity-API gibt es keine strukturierte Update-Kommunikation. Empfehlung: GitHub-Repository beobachten. Auf Commits achten statt Releases.			

SCHWERE	KATEGORIE	FINDING	BETRIFFT
INFO	Authentifizierung	OAuth2 Client Credentials Flow — kein Nutzernamen/Passwort in HA	config_flow.py — CONF_CLIENT_ID, CONF_CLIENT_SECRET
Die Integration verwendet OAuth2 Client Credentials (client_id + client_secret) statt Nutzernamen/Passwort. Das Passwort des Tronity-Accounts wird nicht in HA gespeichert — nur die API-Credentials die in Tronity's Entwicklerportal generiert werden. Gute Praxis. Empfehlung: Kein Handlungsbedarf. Client-Secret kann im Tronity-Portal jederzeit rotiert werden.			

Code-Verbesserungen

TYP	VERBESSERUNG	DATEI
BUG	Token wird nicht gecacht	__init__.py
get_bearer_token() wird bei jedem Update-Zyklus neu aufgerufen. Das Token sollte nach dem ersten Abruf gespeichert und erst bei Ablauf (expires_in) erneuert werden.		
BUG	Kein Fehler-Handling bei None-Token	__init__.py
Wenn bearer_token None ist, wird der API-Request trotzdem abgesetzt. Fehlschlag ohne klare Fehlermeldung.		
QUALITY	iot_class falsch gesetzt	manifest.json
local_polling ist falsch — die Integration pollt ausschliesslich Tronity's Cloud-API.		
QUALITY	async_timeout veraltet	__init__.py
async_timeout.timeout() ist in modernem Python durch asyncio.timeout() ersetzt. Die veraltete API erzeugt Deprecation-Warnings in neueren HA-Versionen.		

Quality Score

Kein Tier erreicht

15 / 55 Kriterien bestanden (7 nicht anwendbar)

Statische Code-Analyse — keine Laufzeitprüfung. Befunde basieren auf dem GitHub-Quellcode zum Zeitpunkt der Analyse.

BRONZE

— Services in <code>async_setup</code> registriert	No custom service actions; sensor/device_tracker entities only
✓ Sinnvolles Poll-Intervall	DataUpdateCoordinator with 60s refresh interval
— Branding-Assets vorhanden	
✗ Gemeinsamer Code in Modulen	Logic distributed across root-level files without shared module
✗ Testabdeckung Config Flow	No tests/ folder in repository
✓ UI-Setup vorhanden	
~ Abhängigkeiten dokumentiert	Dependencies in manifest.json; no requirements.txt; no version pins
— Service Actions dokumentiert	No custom service actions
✓ Beschreibung in Docs	
✓ Installationsanleitung vorhanden	
✗ Entfernungsanleitung vorhanden	No removal instructions in README
✓ Events im richtigen Lifecycle	Coordinator pattern; no direct event subscriptions
✗ Entities haben <code>unique_id</code>	No <code>unique_id</code> set in config flow; duplicates possible
✓ Nutzt <code>has_entity_name</code>	<code>_attr_has_entity_name = True</code> in TronityEntity base class
✗ Nutzt <code>entry.runtime_data</code>	Uses <code>hass.data[DOMAIN]</code> instead of <code>entry.runtime_data</code>
✗ Verbindungstest im Config Flow	Config flow stores credentials without testing API reachability
✗ <code>ConfigEntryNotReady</code> bei Fehler	No <code>ConfigEntryNotReady</code> ; first data fetch failure not handled
✗ Kein Duplikat-Setup möglich	No <code>unique_id</code> → duplicate entries possible

SILBER

— Services werfen korrekte Exceptions	No custom service actions
✓ Unload-Handler implementiert	<code>async_unload_entry</code> implemented
✓ Konfigurationsparameter dokumentiert	Client ID / secret documented
✓ Installationsparameter dokumentiert	
~ Entities werden unavailable markiert	Coordinator sets None on error; entity availability depends on coordinator state
✓ Aktiver Maintainer vorhanden	tronity organisation maintains; official vendor integration
✗ Einmaliges Log bei Unavailability	No explicit unavailability logging pattern
✗ <code>PARALLEL_UPDATES</code> definiert	<code>PARALLEL_UPDATES</code> not defined
✗ Reauth-Flow vorhanden	No <code>async_step_reauth</code> ; expired tokens cause update failures silently
✗ Testabdeckung >95%	No test suite

GOLD

✓	Erstellt HA-Devices	Device registry entry per vehicle
✗	Diagnostics implementiert	No diagnostics.py
—	Discovery aktualisiert Netzwerkinfos	Cloud-only API; no local discovery
—	Automatische Geräteerkennung	Cloud API; device discovery not applicable
~	Datenupdates dokumentiert	60s poll interval mentioned; no detail on what triggers updates
~	Automatisierungsbeispiele vorhanden	example_dashboard.yaml provided; no automation examples
✗	Bekannte Einschränkungen dokumentiert	No known limitations documented
~	Unterstützte Geräte dokumentiert	General EV support mentioned; no explicit device list
✓	Unterstützte Funktionen dokumentiert	Sensor entities listed in README
✗	Troubleshooting-Sektion vorhanden	No troubleshooting section
✗	Anwendungsfälle dokumentiert	No use-case scenarios documented
✗	Devices dynamisch hinzufügbar	Single vehicle per config entry; no dynamic add/remove
✗	EntityCategory gesetzt	EntityCategory not assigned
✓	Device Classes genutzt	SensorDeviceClass used (battery, distance etc.)
✗	Nicht-essentielle Entities deaktiviert	All entities enabled by default
✓	Entity-Übersetzungen vorhanden	strings.json and translations/ present
✗	Exceptions übersetzbar	No HomeAssistantError with translation_key
✗	Icon-Übersetzungen vorhanden	No icons.json
✗	Reconfigurations-Flow vorhanden	No async_step_reconfigure
✗	Repair Issues implementiert	No ir.async_create_issue
—	Veraltete Devices werden entfernt	Single vehicle per entry; no multi-device management

PLATIN

✓	Async-Bibliothek genutzt	aiohttp used for all HTTP requests
✗	Websession injizierbar	aiohttp session created internally; not injectable
✗	Strenge Typisierung	No py.typed; minimal type annotations

ZUSÄTZLICHE KRITERIEN

✗	HTTP-Timeouts konfiguriert	aiohttp requests made without explicit timeout
✗	Abhängigkeiten versionsgepinnt	No requirements.txt; manifest requirements unpinned
~	Keine blanken except-Blöcke	Some broad except blocks around token fetch